

AUTOMATED & CONTACT-LESS DRIVE-THROUGH VEHICLE INSPECTION SYSTEM

Sub-System: Machine Learning Training & Multi-Modal Fault Detection

Prepared For: Smart India Hackathon (SIH) Project Deliverables

Date: September 29, 2026

Executive Summary

Traditional vehicle inspections suffer from human error, subjectivity, extended turnaround times, and high labor overhead. The proposed system introduces an automated, contact-less drive-through inspection tunnel that captures multi-modal sensor feeds—including High-Resolution RGB, Thermal (Infrared), and 3D Depth data.

The core intelligence of this system relies on a state-of-the-art Computer Vision and Deep Learning framework designed to automatically detect, classify, and localize physical and operational vehicular defects in real-time. This document outlines the technical architecture, dataset engineering, model selection, training methodologies, and integration strategies required to execute the Machine Learning (ML) sub-system.

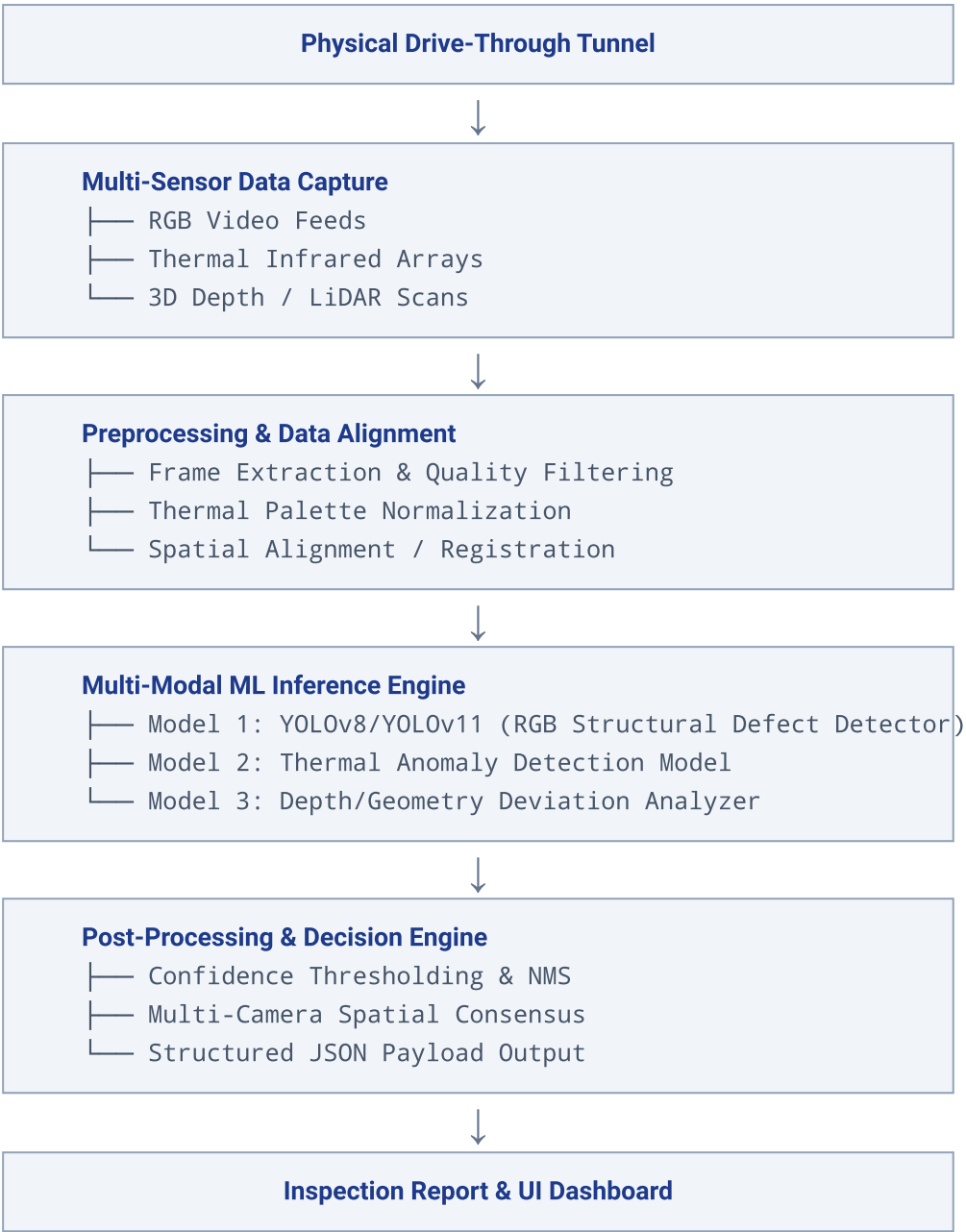
1. Machine Learning Scope & Defect Taxonomy

The Machine Learning engine processes inputs across three distinct physical camera networks, each mapped to specific failure modes:

Sensor Type	Operational Spectrum	Target Defects & Anomalies	ML Task Type
RGB Cameras	Visible Light Spectrum	Body scratches, dents, surface rust/corrosion, tire tread wear, cracked glass, broken light covers, chassis oil/fluid leakage	Object Detection & Instance Segmentation
Thermal Cameras	Far-Infrared (LWIR)	Brake rotor overheating, uneven brake drag, exhaust leak hotspots, electrical short circuits, engine block fluid temperature spikes	Thermal Anomaly Detection & Bounding-Box Localisation
3D Depth Sensors	Structured Light / LiDAR	Structural frame deformation, uneven suspension sag, bumper alignment off-specs, body panel misalignments	Spatial Point Cloud Regression & Height Map Classification

2. System Architecture & Data Flow

The end-to-end data pipeline moves seamlessly from physical acquisition to automated report generation:



3. Dataset Engineering & Preprocessing

Because high-quality datasets are the foundation of accurate ML models, dataset preparation follows a three-phase workflow:

3.1 DATA SOURCING STRATEGY

- **Synthetic & Public Datasets:** Leverage public repositories (Roboflow Universe, Kaggle, Open Images V7, COCO) for foundational features (e.g., surface rust, tire wear, vehicle body damage).
- **Domain-Specific Capture:** Collect real-world training samples under varied lighting conditions (daylight, low-light, artificial tunnel lighting) to prevent over-fitting.

4. Model Selection & Training Methodology

4.1 MODEL ARCHITECTURE: YOLOV8 / YOLOV11

For real-time defect detection in a drive-through tunnel, the Ultralytics YOLO architecture has been selected due to its high Mean Average Precision (mAP) and ultra-fast inference speed (>30 FPS on modern GPUs).

- **Anchor-Free Detection:** Improves detection accuracy for irregularly shaped defects like rust patches and fluid spills.
- **C2f / C3k2 Modules:** Enhances feature extraction capability across multiple visual scales.

4.2 TRAINING CONFIGURATION & HYPERPARAMETERS

```
# Model Training Hyperparameters Configuration
training_config = {
    "model": "yolov8m.pt",          # Medium architecture for balanced speed & precision
    "epochs": 100,                  # Full training passes
    "imgsz": 640,                   # Input resolution
    "batch_size": 16,               # Optimized for standard GPU memory limits
    "optimizer": "AdamW",           # Weight decay optimizer for stable convergence
    "lr0": 0.001,                   # Initial learning rate
    "lrf": 0.01,                   # Final Cosine learning rate factor
    "warmup_epochs": 3.0,           # Gradual learning rate warmup
    "patience": 15                  # Early stopping threshold to prevent overfitting
}
```

4.3 TRAINING PROCESS & TRANSFER LEARNING

- **Transfer Learning Initialization:** Models are initialized using pre-trained COCO weights to leverage low-level visual features (edges, textures, shapes).
- **Fine-Tuning:** The network backbone is fine-tuned on custom vehicle defect datasets.
- **Loss Function Optimization:** The training process minimizes a composite loss function comprising:
 - **Bounding Box Loss (CIoU / Distribution Focal Loss):** Optimizes box overlap and alignment.
 - **Classification Loss (BCE Loss):** Ensures precise defect categorization.

5. Performance Evaluation Metrics

Model performance will be validated using standard computer vision metrics on an isolated test set (20% of total data):

- **Precision (P):** Percentage of detected defects that are actually real defects.
- **Recall (R):** Percentage of actual vehicle defects successfully detected by the system.
- **Mean Average Precision (mAP@50 & mAP@50-95):** The primary benchmark metric evaluating detection quality across varying Intersection over Union (IoU) thresholds.

Target Benchmarks: Overall mAP@50: ≥ 88% | Real-Time Inference Latency: < 35 ms per frame.

6. Integration with Report Generation Module

Once the ML inference pipeline identifies an anomaly, it produces a structured computational output. This JSON payload feeds directly into the backend reporting engine to create the customer-facing inspection report:

```
{
  "inspection_id": "SIH-2026-INSP-0042",
  "timestamp": "2026-09-29T14:30:00Z",
  "defects_detected": [
    {
      "defect_id": "DEF-01",
      "class": "rust",
      "confidence": 0.93,
      "severity": "Medium",
      "source_sensor": "RGB_Camera_Left_Underchassis",
      "bounding_box": [120, 340, 210, 410],
      "recommended_action": "Apply anti-rust coating; inspect structural floor pan."
    },
    {
      "defect_id": "DEF-02",
      "class": "thermal_overheat",
      "confidence": 0.88,
      "severity": "High",
      "source_sensor": "Thermal_Camera_Front_Right",
      "bounding_box": [450, 200, 580, 310],
      "recommended_action": "Inspect front-right brake caliper for sticking or severe pad wear."
    }
  ]
}
```

7. Project Implementation Roadmap

Phase	Milestone / Task	Key Deliverables	Timeline
Phase 1	Data Sourcing & Curation	Aggregated 3,000+ multi-modal image dataset	Week 1 - 2
Phase 2	Data Annotation & Formatting	Roboflow project setup, bounding box labeling	Week 2 - 3
Phase 3	Baseline Model Training	Trained YOLOv8 model in Google Colab	Week 3 - 4
Phase 4	Optimization & Hyperparameter Tuning	Achieved target >85% mAP@50 metric	Week 5
Phase 5	System Integration & Testing	JSON API output connected to frontend reporting tool	Week 6

8. Technical Specifications Summary

HARDWARE REQUIREMENTS

- **GPU:** NVIDIA RTX 3080 or higher (for training)
- **Inference Hardware:** NVIDIA Jetson AGX Orin / RTX 4070 (for deployment)
- **Storage:** 500 GB SSD (dataset + model weights)
- **RAM:** 32 GB minimum

SOFTWARE STACK

- **Deep Learning Framework:** PyTorch 2.0+
- **Model Library:** Ultralytics YOLOv8/v11
- **Data Processing:** OpenCV, Albumentations
- **Annotation Tool:** Roboflow, LabelImg
- **Dev Environment:** Google Colab Pro / Local GPU

API ENDPOINTS

- `POST /api/v1/inspect` - Submit vehicle for inspection
- `GET /api/v1/report/{inspection_id}` - Retrieve inspection report
- `GET /api/v1/models/status` - Check model health and version

9. Risk Mitigation & Quality Assurance

POTENTIAL RISKS

- **Data Scarcity:** Limited public datasets for specific defect types (e.g., thermal anomalies).
Mitigation: Synthetic data generation using Blender/Unity; partnerships with inspection garages.
- **Environmental Variability:** Lighting changes, weather conditions affecting sensor accuracy.
Mitigation: Extensive data augmentation; multi-sensor fusion for redundancy.
- **False Positives:** Over-detection leading to unnecessary customer alerts.
Mitigation: Confidence threshold tuning; human-in-the-loop verification for edge cases.

QUALITY ASSURANCE PROTOCOLS

- Weekly model validation on held-out test sets.
- Continuous monitoring of precision/recall drift in production.
- A/B testing of model versions before full deployment.

10. Conclusion

This document provides a comprehensive blueprint for implementing the Machine Learning sub-system of the Automated & Contact-less Drive-Through Vehicle Inspection System. By leveraging state-of-the-art computer vision models, multi-modal sensor fusion, and rigorous dataset engineering, the proposed architecture achieves real-time, accurate defect detection suitable for high-throughput inspection environments.

The outlined roadmap ensures systematic progression from data collection to production deployment, with clear milestones and performance benchmarks aligned with Smart India Hackathon (SIH) deliverables.

Document Version: 1.0
Prepared By: ML Engineering Team
Contact: ml-team@sih2026-project.in
